WezTerm: Conceptual Architecture Overview

October 6, 2026

Group 17: We can fly

Group members:

Lingwei Huang	21lh66@queensu.ca
Kylin Jin	kylin.j@queensu.ca
Bei Lin	22WN19@queensu.ca
Krystal Sun	23tp43@queensu.ca
Yun Zhang	21yz235@queensu.ca
Hongtian Zhao	22hz16@queensu.ca

Abstract
1. **Introduction**
2. **WezTerm Overview**
   - 2.1. Functionality
   - 2.2. Target Users
   - 2.3. System Boundary
   - 2.4. External Interfaces
   - 2.5. System Evolution
   - 2.6. Developer Responsibilities
3. **Conceptual Architecture**
   - 3.1. Architectural Structure
   - 3.2. Subsystem Responsibilities and Interfaces
   - 3.3. Component Interaction
   - 3.4. Architectural Styles
   - 3.5. Architecture Evaluation
4. **Control Flow and Data Flow**
   - 4.1. Control Flow
   - 4.2. Data Flow
5. **Concurrency**
   - 5.1. Concurrent Processes and Activities
   - 5.2. Communication and Synchronization
   - 5.3. Architectural Implications
6. **Non-Functional Requirements**
   - 6.1. Performance
   - 6.2. Portability
   - 6.3. Modifiability
   - 6.4. Testability
   - 6.5. Reliability and Security
7. **Use Case**
   - 7.1. Use Case 1
   - 7.2. Use Case 2
8. **Data Dictionary and Naming Conventions**
   - 8.1. Data Dictionary
   - 8.2. Naming Conventions
9. **Conclusion and Lessons Learnt**
   - 8.1 Conclusion
   - 8.2 Lessons Learnt
10. **References**
11. **AI Collaboration Report**

Abstract
This report presents the conceptual architecture of WezTerm, a cross-platform terminal emulator and multiplexer primarily written in Rust. WezTerm supports terminal emulation, session multiplexing, remote access, serial communication, GPU-accelerated rendering, and Lua-based configuration across multiple operating systems. Based on the official documentation, we model WezTerm as a hybrid architecture consisting of six major subsystems: Configuration and Event Automation, Graphical User Interface (GUI) Input and Window Integration, Multiplexer and Session Management, Terminal Emulation Core, Rendering and Presentation, and Process and Connection Backends. The multiplexer is identified as the main coordination point because it manages panes, tabs, windows, workspaces, and domain connections while remaining conceptually separate from the graphical interface. The report examines subsystem responsibilities, architectural styles, control and data flow, concurrency, external interfaces, use cases, and non-functional requirements. The analysis shows that WezTerm separates session management, terminal processing, rendering, configuration, and connection handling in a way that supports local and remote workflows, cross-platform portability, customization, and concurrent terminal sessions. This conceptual model provides a basis for later comparison with the concrete architecture recovered from the source code.

1. Introduction
WezTerm is a cross-platform terminal emulator and multiplexer that provides an interactive environment for command-line applications. It supports Linux, macOS, Windows, FreeBSD, and NetBSD, and combines terminal emulation with features such as multiple windows, tabs and split panes, integrated SSH connections, serial communication, GPU-accelerated rendering, and Lua-based configuration. These capabilities make WezTerm suitable for users such as developers, system administrators, and DevOps engineers who may work with several local and remote terminal sessions at the same time. The purpose of this report is to recover and describe the conceptual architecture of WezTerm based on its publicly available documentation. The analysis focuses on the system's major responsibilities and interactions rather than implementation-level classes, functions, or source-code organization. Our model divides WezTerm into six top-level subsystems: Configuration and Event Automation, GUI Input and Window Integration, Multiplexer and Session Management, Terminal Emulation Core, Rendering and Presentation, and Process and Connection Backends. Among these, the multiplexer serves as the central coordination point for terminal sessions and domain connections.The report first introduces WezTerm's functionality, target users, system boundary, evolution, and developer responsibilities. It then presents the conceptual architecture and examines subsystem interactions, architectural styles, control and data flow, concurrency, external interfaces, representative use cases, and major non-functional requirements. The goal is to provide a clear high-level model of how WezTerm is organized and how its major components cooperate before its concrete implementation is examined in A2.

2. WezTerm Overview

2.1. Functionality
WezTerm is a cross-platform terminal emulator and multiplexer primarily written in Rust. Its main functions include terminal emulation, terminal multiplexing and session management, remote access, serial communication, and configuration and customization. It supports multiple operating systems and allows users to organize terminal sessions into windows, tabs, and split panes. WezTerm also provides modern text and graphics rendering, integrated SSH and remote multiplexing, and Lua-based configuration with hot reloading.

2.2. Target Users

WezTerm is particularly useful for developers, system administrators, and DevOps engineers who rely on command-line environments and may need to manage several local or remote terminal sessions. Its SSH support benefits users working with remote systems, serial-port connectivity supports embedded-development workflows, and Lua configuration allows users to customize or automate their terminal environment.

2.3. System Boundary

The WezTerm system boundary includes terminal emulation and multiplexing, the graphical terminal interface, rendering, remote connectivity, serial communication, and configuration management. External entities include the user, operating system, local shells and command-line applications, remote hosts, and physical serial devices. These entities interact with WezTerm through user input and output, terminal I/O, SSH connections, operating-system services, or serial communication.

2.4. External Interfaces

WezTerm exposes several interfaces to its external environment. The graphical user interface supports terminal windows, tabs, split panes, keyboard and mouse interaction, and terminal-output viewing. The Lua configuration interface allows users to control application behavior and appearance. WezTerm also provides network interfaces for remote systems through SSH and a serial interface for communication with external devices.

2.5. System Evolution

WezTerm has expanded significantly since its early releases. Major additions in late 2019 included tab management, configuration hot reloading, SSH and remote multiplexing, serial-port support, inline image display, software-rendering fallback, and improved platform integration. Subsequent releases continued to refine configuration, rendering, SSH behavior, and cross-platform support. Although the frequency of formal releases later decreased, development continues on the main branch.

2.6. Developer Responsibilities

WezTerm was created by Wez Furlong and has been developed with contributions from a broader open-source community. Contributors participate through source-code changes, documentation, bug fixes, design improvements, and technical suggestions. Major development responsibilities include maintaining core functionality, testing and bug correction, cross-platform support, and configuration and documentation resources.

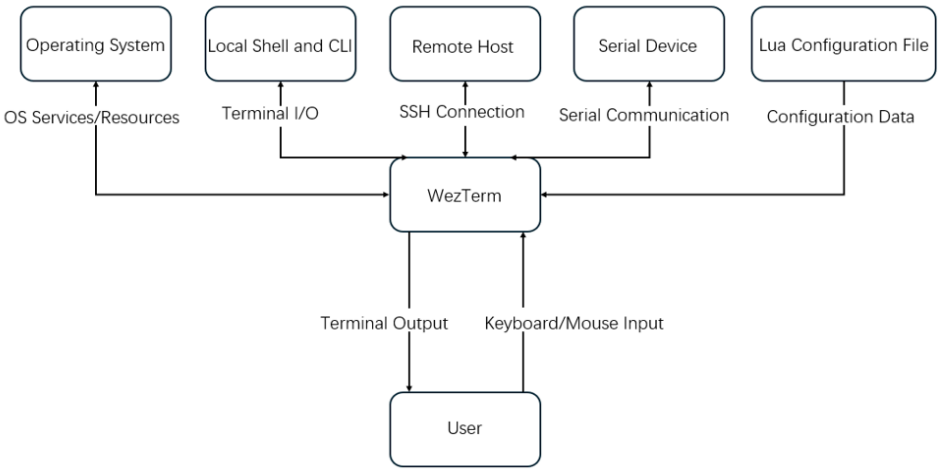


Figure 1. System Context Diagram of WezTerm

3. Conceptual Architecture

3.1. Architectural Structure

We model WezTerm as a hybrid architecture with six top-level subsystems. The Multiplexer and Session Management subsystem is placed at the centre because it coordinates running programs, panes, tabs, windows, workspaces, and domain connections. The remaining subsystems handle configuration, user interaction, terminal emulation, rendering, and local or remote connections. The multiplexer also separates session state from the graphical interface, allowing the same session model to support local shells, remote sessions, and different native window environments.

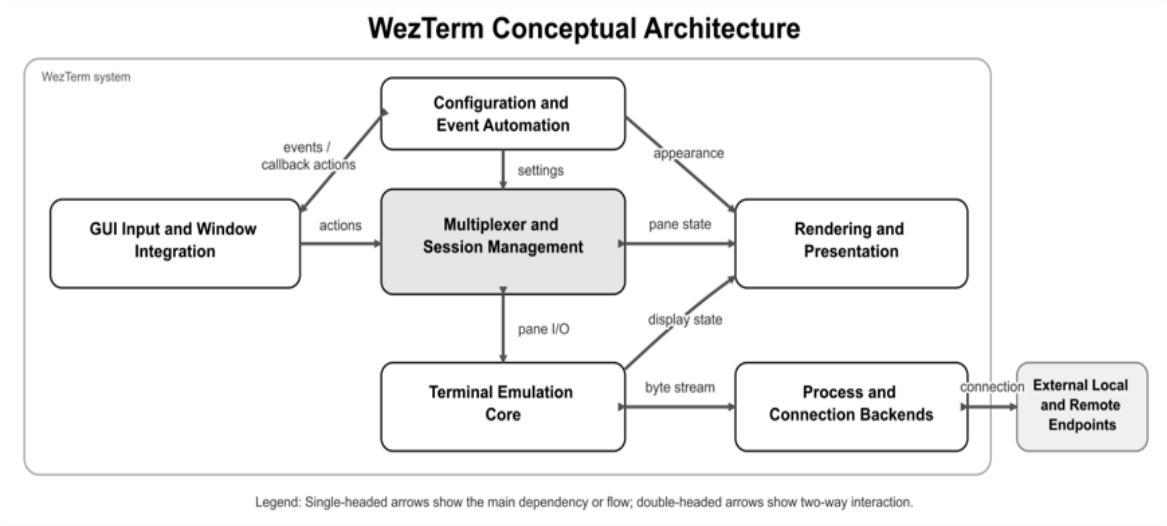


Figure 2. Conceptual architecture and subsystem dependency diagram of WezTerm

3.2. Subsystem Responsibilities and Interfaces

Configuration and Event Automation

Configuration and Event Automation loads Lua configuration, defines key bindings, appearance, domains, and startup behavior, and runs registered event callbacks. Its main interfaces include the Lua configuration table, WezTerm module, event registration, and configuration reload.

GUI Input and Window Integration

GUI Input and Window Integration manages native windows and user interaction, including keyboard, mouse, clipboard, and window events. It connects user actions and GUI objects to the multiplexer.

Multiplexer and Session Management

Multiplexer and Session Management maintains domains, workspaces, windows, tabs, and panes and coordinates session attachment and target-pane selection. Its interfaces include the mux API, domain connections, identifiers, and client-server boundaries.

Terminal Emulation Core

The Terminal Emulation Core parses UTF-8 output and control sequences and updates terminal cells, cursor state, and scrollback. It provides the terminal state consumed by the renderer.

Rendering and Presentation

Rendering and Presentation converts terminal state into visible output, including text, colors, images, and the cursor, using OpenGL, WebGPU, or software rendering.

Process and Connection Backends
Process and Connection Backends connect terminal sessions to local or remote endpoints through mechanisms such as SSH, serial ports, Unix sockets, TLS connections, and mux servers.

3.3. Component Interaction

At startup, the configuration subsystem evaluates the Lua configuration and distributes relevant settings to other subsystems. Configuration can also be hot-reloaded, while registered event handlers may affect startup and runtime behavior.

During terminal interaction, keyboard input is first processed by the GUI. Configured actions may modify panes, tabs, or windows through the multiplexer; otherwise, terminal input is encoded and sent to the process associated with the active pane. Output returns from the local or remote endpoint, is interpreted by the Terminal Emulation Core, and updates the display state used by Rendering and Presentation.

3.4. Architectural Styles

We view WezTerm as a loosely layered system. User actions pass through the GUI and multiplexer, while terminal data moves between panes, connection backends, the terminal core, and the renderer. The layers are not strict because the GUI and Lua callbacks can access responsibilities across layer boundaries, increasing flexibility but also coupling.

WezTerm's remote and socket-based domains also exhibit a client-server style. A GUI client can connect to a multiplexer that manages terminal sessions through mechanisms such as SSH, Unix sockets, or TLS. In addition, the Lua event API resembles an implicit-invocation style because registered callbacks are invoked in response to events.

3.5. Architecture Evaluation

The conceptual model isolates many platform-specific concerns at the GUI, rendering, terminal-communication, and connection boundaries, while keeping the terminal model and multiplexer conceptually separate from native-window operations. These boundaries support portability, remote sessions, configuration, and potential testing separation. However, because the multiplexer acts as the main coordination point for GUI actions, process output, callbacks, and network messages, failures or excessive coupling at this boundary may affect several features.

4. Control Flow and Data Flow

WezTerm separates user interaction, terminal session management, terminal emulation, and graphical rendering. These responsibilities cooperate to transfer user input to running applications and display their output. The following analysis describes the conceptual flow rather than implementation-level function calls.

4.1. Control Flow

Control flow usually begins with a user action received by the GUI. Ordinary terminal input is forwarded to the active pane and then through the Pseudo Terminal (PTY) to the foreground application. Inputs that correspond to WezTerm actions, such as splitting a pane or switching tabs, are instead handled through the multiplexer and the appropriate domain.

The conceptual input path for a local terminal session is:

User → GUI/Input Handling → Active Pane → PTY → Shell/Application

4.2. Data Flow

Terminal data flows in two main directions. Input is encoded by WezTerm and sent through the active pane and PTY to the foreground application. Application output returns through the PTY, is interpreted by the terminal emulator to update terminal state, and is rendered by the GUI.

Application → PTY → Pane/Terminal Emulation → GUI/Renderer → Screen

Remote sessions follow a similar conceptual flow but communicate with a remote multiplexer rather than a local PTY.

5. Concurrency

Concurrency is an important architectural feature of WezTerm, because the application can handle multiple terminal sessions, and communicate with local and remote processes, at the same time.

5.1. Concurrent Processes and Activities

WezTerm must support multiple terminal sessions and local or remote activities concurrently. Different panes may run independent programs while the system simultaneously receives user input, processes terminal output, manages sessions, communicates with remote endpoints, and updates the display. These concurrent activities do not, by themselves, establish a particular internal threading model.

5.2. Communication and Synchronization

Local terminal sessions exchange data with running applications through PTY facilities, while remote multiplexing may use Unix-domain sockets, SSH, or TLS. The multiplexer coordinates session state while the GUI presents that state to the user. Although synchronization is required to keep terminal and session state consistent, the available high-level documentation does not specify the exact threading, scheduling, or synchronization mechanisms; these details should be verified through source-code analysis.

5.3. Architectural Implications

Concurrency requires WezTerm to maintain separate process and terminal state for multiple panes while allowing output to arrive independently of user input. Remote multiplexing introduces additional coordination, latency, and failure concerns. As a result, responsiveness, session consistency, and communication are important architectural concerns.

6. Non-Functional Requirements

In addition to supporting terminal interaction and multiplexing, WezTerm's architecture must address several non-functional requirements (NFRs). The requirements discussed below are identified from its documented functionality and architectural responsibilities; they should not be interpreted as a formally published requirements specification.

6.1. Performance

Performance is important for input responsiveness, terminal emulation, and rendering. WezTerm supports GPU-accelerated OpenGL and WebGPU rendering as well as software rendering, allowing presentation to adapt to different hardware environments. The architecture must also process output from multiple active terminal sessions efficiently.

6.2. Portability

WezTerm supports Windows, macOS, Linux, and other operating systems, making portability an important architectural concern. Platform differences affect both terminal communication and rendering; for example, Unix-like systems typically use PTYs while Windows provides ConPTY. Separating common terminal responsibilities from platform-specific mechanisms helps support multiple environments.

6.3. Modifiability

Lua-based configuration, customizable key bindings, and hot reloading allow users to modify many behaviors without changing source code. Separating session management from execution domains also provides a conceptual basis for supporting different local and remote environments. The actual implementation cost of architectural changes, however, cannot be determined from the conceptual model alone.

6.4. Testability

Separating terminal emulation, process communication, rendering, and remote connectivity creates potential testing boundaries because each responsibility has observable inputs and outputs. Some behaviors still require integration testing across components or platform-specific facilities. Actual automated test coverage must be evaluated from the source repository.

6.5. Reliability and Security

Reliability is particularly important for remote terminal sessions. Remote multiplexing can allow sessions to persist independently of a local GUI connection, although this depends on the selected connection mode and configuration. Security is also important because remote connectivity may use SSH or TLS-protected multiplexing, with overall security depending on deployment configuration and environment.

7. Use Cases

7.1. Use Case 1: Execute a Command in a Local Terminal

UC-01 describes command execution in an active local terminal session. The user enters a command through the Graphical User Interface (GUI), which forwards the input to the active terminal pane. The pane sends the input through the Pseudo Terminal (PTY) to the shell, which executes the command and returns output through the same path. The terminal emulator processes the output, updates terminal state, and the GUI displays the result. If the command is invalid, the shell returns an error message instead. The terminal session remains active after the interaction.

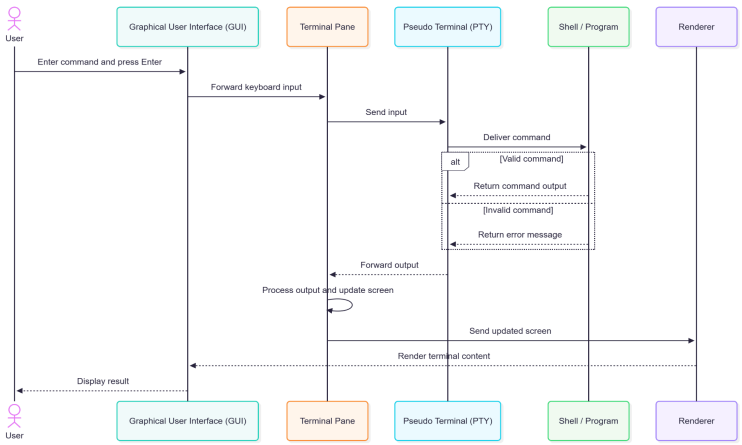


Figure 3. Sequence Diagram for UC-01: Execute a Command in a Local Terminal

7.2. Use Case 2: Terminal Input and Output Processing

While UC-01 presents command execution from the user's perspective, UC-02 focuses on the internal conceptual processing of terminal input and output. The terminal pane forwards input to the multiplexer, which writes it to the PTY and associated shell or program. Program output returns through the PTY, after which the terminal parser interprets the output and updates the screen model. The multiplexer then notifies the presentation side, and the renderer displays the updated terminal content.

Error output and escape sequences follow the same processing path, while a closed PTY prevents the pane from receiving further process output. The session remains active unless the PTY or process terminates.

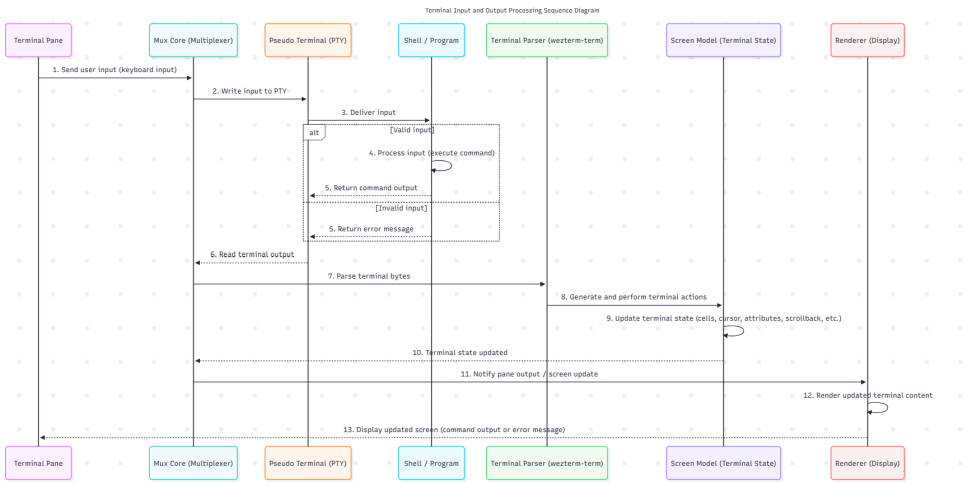


Figure 4. Sequence diagram for UC-02: Terminal Input and Output Processing

8. Data Dictionary and Naming Conventions

8.1 Data Dictionary

Term	Definition
GUI (Graphical User Interface)	The presentation and interaction layer through which users access terminal windows, panes, keyboard input, mouse actions, and other graphical functions.
Multiplexer	The subsystem responsible for managing domains, workspaces, windows, tabs, panes, and session attachment.
Pane	An individual terminal session within WezTerm that is associated with terminal state and a running local or remote process.
Tab	A logical grouping of one or more terminal panes within a WezTerm window.
Workspace	A higher-level grouping used by the multiplexer to organize windows and terminal sessions.

Domain	The execution or connection context in which terminal sessions are created, such as local or remote multiplexing environments.
PTY (Pseudo Terminal)	The interface used in a local terminal session to exchange input and output between WezTerm and a foreground shell or application.
Terminal Emulation Core	The subsystem that parses terminal output and control sequences and updates terminal cells, cursor state, and scrollback.
Rendering and Presentation	The subsystem that converts terminal state into visible output such as text, colors, images, and the cursor.
Process and Connection Backends	The subsystem that connects terminal sessions to local or remote endpoints using mechanisms such as SSH, serial ports, Unix sockets, TLS, and mux servers.

8.2 Naming Conventions

Use cases are identified using the format UC-01, UC-02, and so on. Major component and subsystem names use Title Case, such as Terminal Emulation Core and Multiplexer and Session Management. Abbreviations are written in full when they first appear, followed by the abbreviation in parentheses, such as Graphical User Interface (GUI) and Pseudo Terminal (PTY). The same component names are used consistently across the architecture description, use cases, and sequence diagrams. Interaction labels in sequence diagrams use verb phrases, such as Send Input, Return Output, and Update Terminal State.

9. Conclusion and Lessons Learnt

9.1. Conclusion

Our conceptual analysis models WezTerm as a hybrid architecture with six major subsystems: Configuration and Event Automation, GUI Input and Window Integration, Multiplexer and Session Management, Terminal Emulation Core, Rendering and Presentation, and Process and Connection Backends. The multiplexer serves as the main coordination point by managing domains, workspaces, windows, tabs, panes, and session attachment. Its separation from the graphical interface is especially important because it supports both local and remote terminal workflows while keeping session management conceptually independent from presentation.

The architecture also separates user interaction, terminal processing, rendering, and connection handling. User input is received by the GUI and either triggers a WezTerm action or is forwarded to an active terminal session, while application output is processed by the terminal emulation subsystem and rendered back to the user. This structure supports concurrent sessions and addresses important quality concerns such as performance, portability, modifiability, testability, reliability, and security. Overall, the conceptual model provides a foundation for comparing WezTerm's documented architecture with its concrete implementation in A2.

9.2. Lessons Learned

One important lesson from this assignment was that conceptual architecture recovery involves grouping documented features into broader architectural responsibilities rather than

simply listing them. We also found that consistent terminology is essential when multiple group members contribute to the same report. Terms such as GUI, multiplexer, pane, terminal emulator, renderer, and PTY must be used consistently across diagrams, use cases, and analysis.

We also learned to distinguish documented architectural behavior from implementation details that cannot yet be confirmed. For example, the documentation supports concurrent terminal activities but does not fully specify internal threading or synchronization mechanisms, so these details should be verified later through source-code analysis. In future deliverables, we would define shared terminology, subsystem boundaries, and report structure earlier to reduce duplication and simplify integration.

References

Angela. (2026, May 14). What is WezTerm used for? WezTerm. https://wezterm.com/2026/05/14/what-is-wezterm-used-for/

Furlong, W. (n.d.). wez. GitHub. Retrieved October 3, 2026, from https://github.com/wez

MyNixOS. (n.d.). wezterm. Retrieved October 3, 2026, from https://mynixos.com/nixpkgs/package/wezterm

WezTerm. (n.d.-a). Building from source. Retrieved October 3, 2026, from https://wezterm.org/install/source.html

WezTerm. (n.d.-b). Change log. Retrieved October 3, 2026, from https://wezterm.org/changelog.html

WezTerm. (n.d.-c). Configuration. Retrieved October 3, 2026, from https://wezterm.org/config/files.html

WezTerm. (n.d.-d). Features. Retrieved October 3, 2026, from https://wezterm.org/features.html

WezTerm. (n.d.-e). front_end = "OpenGL". Retrieved October 3, 2026, from https://wezterm.org/config/lua/config/front_end.html

WezTerm. (n.d.-f). Keyboard encoding. Retrieved October 3, 2026, from https://wezterm.org/config/key-encoding.html

WezTerm. (n.d.-g). Multiplexing. Retrieved October 3, 2026, from https://wezterm.org/multiplexing.html

WezTerm. (n.d.-h). Pane object. Wez's Terminal Emulator. Retrieved October 4, 2026, from https://wezterm.org/config/lua/pane/

WezTerm. (n.d.-i). serial_ports. Retrieved October 3, 2026, from https://wezterm.org/config/lua/config/serial_ports.html

WezTerm. (n.d.-j). SSH. Retrieved October 3, 2026, from https://wezterm.org/ssh.html

WezTerm. (n.d.-k). wezterm.mux module. Retrieved October 3, 2026, from https://wezterm.org/config/lua/wezterm.mux/index.html

WezTerm. (n.d.-l). wezterm.on (event_name, callback). Retrieved October 3, 2026, from https://wezterm.org/config/lua/wezterm/on.html

WezTerm. (n.d.-m). What is a terminal?. Wez's Terminal Emulator. Retrieved October 4, 2026, from https://wezterm.org/what-is-a-terminal.html

AI Collaboration Report

AI Member Profile and Selection Process

Our group used ChatGPT with the GPT-5.6 Sol model as the virtual AI teammate for Assignment 1. We selected ChatGPT because the project required us to organize technical documentation, recover a conceptual architecture, review diagrams and use cases, and integrate independently written sections into a consistent report. We mainly used AI as an organization and review tool rather than as a substitute for architectural analysis. Group members collected information from official WezTerm documentation, prepared drafts for their assigned sections, and then used AI to organize, revise, or evaluate that work. Final architectural decisions remained with the group.

Task Assigned to the AI Teammate

We assigned AI the following tasks: organize notes from the WezTerm documentation and changelog because AI could group related evidence efficiently; review the system boundary and six proposed subsystems to identify overlap or missing responsibilities; check component relationships and compare diagrams with the written analysis; review use cases and sequence diagrams for missing actors, flows, and consistent terminology; identify duplicated content and revise grammar during final integration; and produce an initial conceptual architecture diagram from the group's six-subsystem model. These tasks involved organization, comparison, and consistency checking. The group selected the evidence, verified technical claims, revised the AI-assisted diagram, and made all final architectural decisions. It also assisted with grammar, sentence clarity, and the integration of the Abstract, Introduction, Conclusion, Lessons Learnt, use cases, and architecture sections.

These tasks were appropriate for AI because they primarily involved organization, comparison, consistency checking, and language revision. The group remained responsible for interpreting the documentation and deciding which architectural claims were sufficiently supported.

Interaction Protocol and Prompting Strategy

AI use was distributed across the group rather than handled by a single prompt engineer. Members prepared their assigned sections and used ChatGPT to review or improve them. During final integration, the group leader also used AI to compare independently written sections, reduce duplication, standardize terminology, and review the overall coherence of the report.

The prompting process became more specific over time. Early prompts sometimes produced unnecessary implementation details or expressed architectural interpretations too confidently. We therefore added constraints requiring the AI to rely on provided notes and official WezTerm documentation, distinguish documented facts from architectural interpretation, and remain at the conceptual rather than source-code level.

One representative prompt used during the architecture analysis asked the AI to review the six proposed conceptual subsystems and determine whether any responsibilities overlapped or were missing. The prompt was: "We currently divide WezTerm into six conceptual subsystems: Configuration and Event Automation, GUI Input and Window Integration, Multiplexer and Session Management, Terminal Emulation Core, Rendering and Presentation, and Process and Connection Backends. Based only on the official WezTerm documentation and the notes provided below, examine whether these categories overlap or omit an important top-level responsibility." We later refined this type of prompt by requiring

the AI to distinguish documented facts from architectural interpretations and to avoid implementation-level details.
For the use cases, prompts were similarly constrained to check completeness, grammatical accuracy, terminology consistency, and alignment with the conceptual architecture while avoiding unsupported implementation-level details.

Validation and Quality Control Procedures
All AI-assisted content was reviewed before inclusion in the final report. Technical claims were checked against official WezTerm documentation, including sources on configuration, multiplexing, terminal behavior, rendering, events, and system evolution. AI-organized changelog information was manually compared with the original source, and system-boundary suggestions were checked against the final System Context Diagram.
For architecture-related content, unsupported implementation assumptions were removed, especially for concurrency, threading, and synchronization, where the high-level documentation does not reveal the complete internal implementation. For use cases and sequence diagrams, members compared the diagrams with the written flows and checked component names against the conceptual architecture. AI-assisted compression and language revisions were also manually reviewed to ensure that technical meaning was not changed or lost.

Quantitative Contribution to the Final Deliverable
The group estimates that AI contributed 30% of the final A1 deliverable. This contribution primarily consisted of organizing research notes and changelog information, reviewing subsystem boundaries and system relationships, checking consistency between diagrams and written analysis, assisting with use-case and sequence-diagram structure, improving terminology and grammar, and helping integrate independently written sections.
The remaining work was completed by the group, including reading the original documentation, selecting evidence, deciding the final conceptual architecture, creating and revising diagrams, verifying technical claims, managing references, and deciding which AI suggestions to accept or reject.

Reflection on Human-AI Team Dynamics
AI made the organization, review, and integration stages more efficient. It was particularly useful for identifying duplicated explanations, inconsistent subsystem terminology, missing use-case elements, and differences between diagrams and written descriptions. It also helped organize chronological information such as system evolution.
However, AI also created additional validation work. Some responses included implementation details outside the scope of A1 or presented architectural interpretations more confidently than the available documentation supported. These issues required the group to refine prompts, verify claims against primary sources, and remove unsupported material. We learned that AI is most useful when it is given a clearly defined task, relevant source material, and explicit limits on what it may infer. In future deliverables, we would establish shared terminology and prompting guidelines earlier while continuing to keep source verification and final architectural decisions under human control.